CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

# Experiment – 3

**Name:** Aariz Raza
**UID:** 24BAI71000
**Branch:** CSE AIML (CS221)
**Section/Group:** 24AIT_NTPP-3[B]
**Semester:** 5th
**Date of Performance:** 22/07/2026
**Subject:** Full Stack-II
**Subject Code:** 24CSP-337

## EXPERIMENT – 3.1

1. **AIM:** To design and implement a secure authentication system using JSON Web Tokens (JWT) for user login and session management.

2. **OBJECTIVE:**

- To understand authentication mechanisms in web applications.
- To implement token-based authentication using JWT.
- To manage user sessions in a stateless architecture.
- To handle token storage and validation securely.

3. **SOFTWARE REQUIREMENTS:**

1. React.js
2. Node.js (Optional)
3. JWT Libraries (jsonwebtoken)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. npm (Node Package Manager)

4. **THEORY**

Authentication is the process of verifying the identity of a user before allowing access to protected resources. In traditional web applications, user sessions are maintained on the server using session IDs, which increases server-side storage and management. JSON Web Token (JWT) provides a stateless authentication mechanism, where the server generates a signed token after successful login. The client stores this token and sends it with every subsequent request for authentication.

A JWT consists of three parts: Header, which specifies the signing algorithm; Payload, which contains user information or claims; and Signature, which ensures the token has not been modified. During authentication, the server verifies the token before granting access to protected resources.

In this experiment, a login interface is created where user credentials are validated (using mock or static data). After successful authentication, a JWT token is generated or simulated and stored in localStorage or sessionStorage. The stored token is attached to future requests and can be decoded to retrieve user information. This experiment introduces the concepts of stateless authentication, secure token storage, session management, and JWT-based authorization, which are widely used in modern web applications.

## 5. IMPLEMENTATION/PROCEDURE

1. Create a React login form.
2. Accept username and password from the user.
3. Validate user credentials (mock/static).
4. Generate or simulate a JWT token after successful login.
5. Store the token in localStorage or sessionStorage.
6. Retrieve the stored token when required.
7. Attach the token with API requests.
8. Decode the token to extract user information.
9. Verify authentication before granting access.
10. Test the login and session management functionality.

## 6. CODE:

```
import { useState } from "react";

function App() {

 const [username, setUsername] = useState("");

 const [password, setPassword] = useState("");

 const login = () => {

  if (username === "admin" && password === "1234") {

   const token = "sample-jwt-token";

   localStorage.setItem("token", token);

   alert("Login Successful");

  } else {

   alert("Invalid Credentials");

  }

 };

 return (

  <div style={{ textAlign: "center", marginTop: "50px" }}>

   <h2>JWT Login Demo</h2>

   <input

    type="text"

    placeholder="Username"

    value={username}

    onChange={(e) => setUsername(e.target.value)}
```

```
      />
      <br /><br />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <br /><br />
      <button onClick={login}>Login</button>
    </div>
  );
}
 export default App;
```

## 7. OUTPUT

The application provides a login interface where users enter their credentials. If the credentials are valid, a JWT token is generated (or simulated) and stored in localStorage, allowing the user to maintain a secure session. The stored token can be used to authenticate future requests without creating server-side sessions.

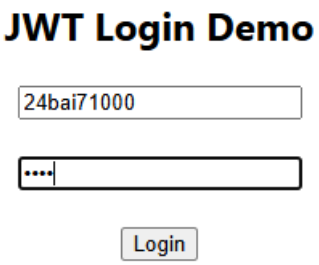


**Fig 3.1:** JWT Authentication System

## 8. LEARNING OUTCOME

1. Understand the concept of authentication in web applications.
2. Learn the working of JSON Web Tokens (JWT).
3. Implement token-based authentication.
4. Manage user sessions using stateless architecture.
5. Store authentication tokens securely.
6. Validate and decode JWT tokens.
7. Improve application security using token-based authentication.
8. Develop secure React applications with JWT authentication.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

# EXPERIMENT – 3.2

## 1. AIM: To implement Role-Based Access Control (RBAC) and secure application routes based on user permissions.

## 2. OBJECTIVE:

- To understand authorization mechanisms in applications.
- To implement Role-Based Access Control (RBAC).
- To protect routes based on user roles.
- To dynamically render UI elements based on user permissions.

## 3. SOFTWARE REQUIREMENTS:

- React.js
- React Router
- Context API / Redux
- Node.js
- Visual Studio Code
- Google Chrome / Microsoft Edge
- npm (Node Package Manager)

## 4. THEORY

Authorization is the process of determining what actions an authenticated user is allowed to perform within an application. Unlike authentication, which verifies a user's identity, authorization decides the level of access granted to the user. One of the most commonly used authorization models is Role-Based Access Control (RBAC), where users are assigned specific roles such as Admin, Editor, or Viewer. Each role has predefined permissions that determine which features or pages the user can access.

In this experiment, RBAC is implemented using React.js and React Router. After a user logs in, their role is stored in the application's authentication state using Context API or Redux. Protected routes are created so that only authorized users can access specific pages. For example, an Admin can access all pages, an Editor can edit content but cannot manage users, while a Viewer can only view information. The application also uses conditional rendering to display buttons, menus, and pages according to the user's role. Unauthorized users attempting to access restricted pages are automatically redirected. This experiment demonstrates authorization, route protection, role management, conditional rendering, and secure navigation, which are essential concepts in modern web application development.

## 5. IMPLEMENTATION/PROCEDURE

1. Create a React project.
2. Define user roles (Admin, Editor, Viewer).
3. Store the user's role in Context API or Redux.
4. Configure React Router for application routing.
5. Create protected routes based on user roles.
6. Check user permissions before rendering pages.
7. Display or hide UI elements according to the assigned role.

8. Redirect unauthorized users to the login or home page.
9. Test access with different user roles.
10. Verify secure navigation and route protection.

## 6. CODE

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";

const role = "Admin"; // Change to Editor or Viewer

function AdminPage() {
  return <h2>Welcome Admin</h2>;
}

function Home() {
  return <h2>Home Page</h2>;
}

function ProtectedRoute({ children }) {
  return role === "Admin" ? children : <Navigate to="/" />;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />

        <Route
          path="/admin"
          element={
            <ProtectedRoute>
              <AdminPage />
            </ProtectedRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  );
}
```

export default App;

## 7. OUTPUT (SCREENSHOT OF WEB + explain)

The application successfully implements Role-Based Access Control (RBAC). Users can access pages only if they have the required role. Authorized users can navigate to protected routes, while unauthorized users are automatically redirected. The user interface also changes dynamically based on the assigned role, improving application security.

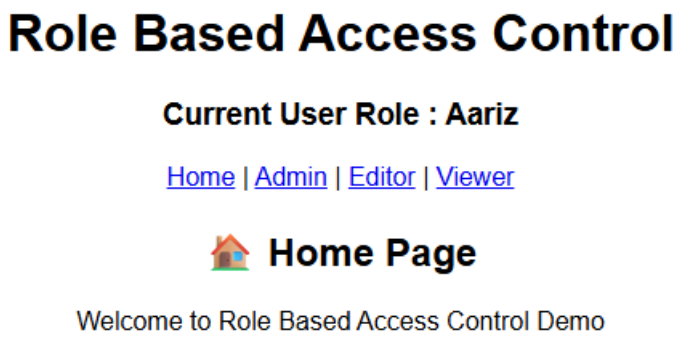


**Fig 3.2:** Role-Based Access Control (RBAC)

## 8. LEARNING OUTCOME

- Understand the concept of authorization in web applications.
- Learn the working of Role-Based Access Control (RBAC).
- Implement role-based authentication and authorization.
- Protect application routes using React Router.
- Dynamically render UI elements based on user roles.
- Restrict unauthorized access to protected resources.
- Improve application security through role management.
- Develop secure React applications using RBAC.